Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 5

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

« Розробка багатовіконного інтерфейсу

користувача для графічного редактора об’єктів »

Виконав:

студент групи ІМ-53
Брагар Артем Сергійович
Номер в списку - 2

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі MS Visual Studio C++ проект Desktop Application з ім’ям Lab5.

2. Написати вихідний текст програми згідно варіанту завдання.

3. Скомпілювати вихідний текст і отримати виконуваний файл програми.

4. Перевірити роботу програми. Налагодити програму.

5. Проаналізувати та прокоментувати результати та вихідний текст програми.

6. Оформити звіт.

Завдання згідно варіанту:

1. Для усіх варіантів завдань необхідно дотримуватися вимог та положень, викладених вище у порядку виконання роботи та методичних рекомендаціях.
2. об’єкт класу Editor на основі класичної реалізації Singleton.
3. Усі кольори та стилі геометричних форм – як у попередньої лаб. роботі №4.
4. Запрограмувати вікно таблиці. Для його відкриття та закриття передбачити окремий пункт меню. Вікно таблиці повинно автоматично закриватися при виході з програми.
5. Вікно таблиці – немодальне вікно діалогу. Таблиця повинна бути запрограмована як клас у окремому модулі. Інтерфейс модуля у вигляді оголошення класу таблиці
6. Запрограмувати запис файлу множини об’єктів, що вводяться
7. Оголошення класів для усіх типів об'єктів робити у окремих заголовочних файлах, а визначення функцій членів – у окремих файлах *.cs. Таким чином, програмний код для усіх наявних типів об'єктів розподілюється по множині окремих модулів.
8. Ієрархія класів та побудова модулів повинні бути зручними для можливостей додавання нових типів об'єктів без переписування коду вже існуючих модулів.

9. У звіті повинна бути схема успадкування класів – діаграма класів. Побудувати діаграму класів засобами Visual Studio.
11. Бонуси-заохочення, які можуть суттєво підвищити оцінку лабораторної роботи. Оцінка підвищується за виконання кожного пункту, з наведених нижче:
1). Якщо у вікні таблиці буде передбачено, щоб користувач міг виділити курсором рядок таблиці і відповідний об’єкт буде якось виділятися на зображенні у головному вікні.
2). Якщо у вікні таблиці користувач може виділити курсором рядок таблиці і відповідний об’єкт буде вилучено з масиву об’єктів.
89
При виконанні бонусів 1 та 2 забороняється робити для цього нові залежності модуля my_table від інших .cpp файлів. Тоді як надіслати повідомлення (наприклад, про виділення користувачем якогось рядка таблиці) від вікна таблиці клієнту цього вікна (наприклад, коду головного файлу .cpp)? Підказки можна знайти у матеріалі лекції стосовно технології Callback, а також патернів Observer, Listener.
3). Якщо програма не тільки записує у файл опис множини об’єктів, а ще й здатна завантажити такий файл і відобразити відповідні об’єкти у головному вікні та вікні таблиці

Файли Лабораторної
Файли розширення(.cs)

ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape

```
using System.Drawing.Drawing2D;

namespace Lab5;

public class ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape
    : Shape
    {
    private const int Radius = 16;
    private EllipseShape firstCircle = new EllipseShape();
    private EllipseShape secondCircle = new EllipseShape();
```

```
        private LineShape line = new LineShape();


    private void DrawEnclosingCircle(Graphics g, Point center,
                    EllipseShape circle)
                                {
      circle.StartPoint = new Point(center.X - Radius, center.Y -
                            Radius);
    circle.EndPoint = new Point(center.X + Radius, center.Y + Radius);
                            circle.Show(g);
                                }
              public override void Show(Graphics g)
                                {
          line.OutlinePen.DashStyle = OutlinePen.DashStyle;
       firstCircle.OutlinePen.DashStyle = OutlinePen.DashStyle;
      secondCircle.OutlinePen.DashStyle = OutlinePen.DashStyle;
                    line.StartPoint = StartPoint;
                      line.EndPoint = EndPoint;
                            line.Show(g);
          DrawEnclosingCircle(g, StartPoint, firstCircle);
          DrawEnclosingCircle(g, EndPoint, secondCircle);
                                }
             public override void SetColor(Color color)
                                {
                        base.SetColor(color);
                        line.SetColor(color);
                    firstCircle.SetColor(color);
                   secondCircle.SetColor(color);
                                }
```

```
        public override void Dispose()
        {
            firstCircle.Dispose();
            secondCircle.Dispose();
            line.Dispose();
            base.Dispose();
        }
    }
```

CubeShape

```
namespace Lab5;
public class CubeShape : Shape
{
    private RectShape frontRect = new RectShape();
    private RectShape backRect = new RectShape();
    private LineShape[] lineShapes = new LineShape[4];
    public CubeShape()
    {
        for(int i = 0; i < lineShapes.Length; i++)
        {
            lineShapes[i] = new LineShape();
        }
    }

    public override void Show(Graphics g)
    {
        frontRect.OutlinePen.DashStyle = OutlinePen.DashStyle;
```

```
            backRect.OutlinePen.DashStyle = OutlinePen.DashStyle;
                  for(int i = 0; i < lineShapes.Length; i++)
                                        {
                lineShapes[i].OutlinePen.DashStyle = OutlinePen.DashStyle;
                                        }
                    int left = Math.Min(StartPoint.X, EndPoint.X);
                   int right = Math.Max(StartPoint.X, EndPoint.X);
                    int top = Math.Min(StartPoint.Y, EndPoint.Y);
                  int bottom = Math.Max(StartPoint.Y, EndPoint.Y);
                             int width = (right - left) ;
                             int height = (bottom - top);
                     int depth = (Math.Min(width, height) * 1/3);


         frontRect.StartPoint = new Point(((left + (right - depth)) / 2),
                      (((top + depth) + bottom) / 2));
             frontRect.EndPoint = new Point((right - depth) , bottom);


        backRect.StartPoint = new Point((right + (left + depth)) / 2, (top
                        + (bottom - depth)) / 2 );
              backRect.EndPoint = new Point(right, bottom - depth);




             lineShapes[0].StartPoint = new Point(left, top + depth);
              lineShapes[0].EndPoint = new Point(left + depth, top);


         lineShapes[1].StartPoint = new Point(right - depth, top + depth);
                  lineShapes[1].EndPoint = new Point(right, top);
```

```
            lineShapes[2].StartPoint = new Point(left, bottom);
        lineShapes[2].EndPoint = new Point(left + depth, bottom - depth);


          lineShapes[3].StartPoint = new Point(right - depth, bottom);
           lineShapes[3].EndPoint = new Point(right, bottom - depth);


                              frontRect.Show(g);
                              backRect.Show(g);
                    for(int i = 0; i < lineShapes.Length; i++)
                                        {
                              lineShapes[i].Show(g);
                                        }
                                   }
                    public override void SetColor(Color color)
                                   {
                               base.SetColor(color);
                    for(int i = 0; i < lineShapes.Length; i++)
                                        {
                            lineShapes[i].SetColor(color);
                                        }
                            frontRect.SetColor(color);
                            backRect.SetColor(color);
                                   }
                        public override void Dispose()
                                   {
                               frontRect.Dispose();
                               backRect.Dispose();
```

```
            for(int i = 0; i < lineShapes.Length; i++)
                {
                lineShapes[i].Dispose();
                }
            base.Dispose();
            }
        }
```

EllipseShape

```
namespace Lab5;
public class EllipseShape : Shape
    {
    public Brush FillBrush = new SolidBrush(Color.Cyan);
    public override void Show(Graphics g)
        {
        int radX = Math.Abs(EndPoint.X - StartPoint.X);
        int radY = Math.Abs(EndPoint.Y - StartPoint.Y);
        int x = Math.Min(StartPoint.X, EndPoint.X);
        int y = Math.Min(StartPoint.Y, EndPoint.Y);
    if(!IsPreview) {g.FillEllipse(FillBrush, x, y, radX, radY);}
        g.DrawEllipse(OutlinePen, x, y, radX, radY);
        }
    public override void Dispose()
        {
        FillBrush.Dispose();
        base.Dispose();
        }
```

```
}
```

LineShape

```
namespace Lab5;

public class LineShape : Shape

{

public override void Show(Graphics g)

{

g.DrawLine(OutlinePen, StartPoint, EndPoint);

}

}
```

PointShape

```
namespace Lab5;

public class PointShape : Shape

{

public SolidBrush FillBrush;

public PointShape()

{

FillBrush = new SolidBrush(OutlinePen.Color);

}

public override void Show(Graphics g)

{

int width = 4;

int height = 4;

int x = StartPoint.X - (width / 2);

int y = StartPoint.Y - (height / 2);

g.FillEllipse(FillBrush, x, y, width, height);

}
```

```
        public override void SetColor(Color color)
        {
            FillBrush.Color = color;
        }
        public override void Dispose()
        {
            FillBrush.Dispose();
            base.Dispose();
        }
    }
```

RectShape

```
namespace Lab5;
public class RectShape : Shape
{
    public override void Show(Graphics g)
    {
        int dx = Math.Abs(EndPoint.X - StartPoint.X);
        int dy = Math.Abs(EndPoint.Y - StartPoint.Y);
        int width = dx * 2;
        int height = dy * 2;
        int x = StartPoint.X - dx;
        int y = StartPoint.Y - dy;
        g.DrawRectangle(OutlinePen, x, y, width, height);
    }
}
```

Shape

```
namespace Lab5;
```

```
public abstract class Shape : IDisposable
{
    public Point StartPoint = new Point();
    public Point EndPoint = new Point();
    public Pen OutlinePen = new Pen(Color.Black);
    public bool IsPreview {get; set;}
    public string? Key {get; set;}
    public abstract void Show(Graphics g);
    public virtual void Dispose()
    {
        OutlinePen.Dispose();
    }
    public virtual void SetColor(Color color)
    {
        OutlinePen.Color = color;
    }
}
```

Descriptor

```
namespace Lab5;
public class Descriptor
{
    public string Name {get;}
    public string Key {get;}
    public Bitmap Icon {get;}
    public Func<Shape> Create {get;}
```

```
    public Descriptor(string name, string key,Bitmap icon, Func<Shape>
                                create)
                                    {
                                     Name = name;
                                      Key = key;
                                     Icon = icon;
                                    Create = create;
                                    }
                                   }
```

Editor

```
                            namespace Lab5;
                        using System.Drawing.Drawing2D;
                       public class Editor : IDisposable
                                    {
                            public enum OnMouseUpResult
                                    {
                                     ShapeAdded,
                                     ArrayIsFull,
                                    NothingHappend
                                    }
   private Descriptor currentShapeType = ShapesDescriptor.Descriptors[0];
                       private const int MaxShapes = 103;
                           private int shapeCount = 0;
                            private int? selectedRow;
                         private bool isDrawing = false;
                           private Point currentStart;
                            private Point currentEnd;
```

```
                private static Editor? instance;

                 public static Editor Instance

                             {

                              get

                                {

                          if(instance == null)

                                    {

                          instance = new Editor();

                                    }

                            return instance;

                                  }

                             }

                    private Shape[] shapes;

                        private Editor()

                             {

                     shapes = new Shape[MaxShapes];

                             }

                      public int ShapeCount

                             {

                              get

                                {

                           return shapeCount;

                                  }

                             }

              public void OnMouseDown(Point point)

                             {

                        isDrawing = true;
```

```
                currentStart = point;
                currentEnd = point;
            }
        public bool OnMouseMove(Point point)
            {
                if (isDrawing)
                    {
                currentEnd = point;
                    }
                return isDrawing;
            }
         public OnMouseUpResult OnMouseUp()
            {
                if (!isDrawing)
                    {
           return OnMouseUpResult.NothingHappend;
                    }

                isDrawing = false;

              Shape newShape = CreateShape();
            newShape.StartPoint = currentStart;
              newShape.EndPoint = currentEnd;

                if (!AddShape(newShape))
                    {
                newShape.Dispose();
```

```
            return OnMouseUpResult.ArrayIsFull;
                        }
            return OnMouseUpResult.ShapeAdded;
                        }
             private Shape CreateShape()
                        {
        Shape shape = currentShapeType.Create();
            shape.Key = currentShapeType.Key;
                    return shape;
                        }
         public void DrawShapes(Graphics g)
                        {
          for (int i = 0; i < shapeCount; i++)
                        {
                    if(i == selectedRow)
                            {
                shapes[i].SetColor(Color.Blue);
                        }
                    shapes[i].Show(g);
              shapes[i].SetColor(Color.Black);
                        }
                    if (isDrawing)
                        {
                       DrawTemp(g);
                        }
                    }
         private void DrawTemp(Graphics g)
```

```
                {
            using (Shape previewShape = CreateShape())
                {
                    previewShape.IsPreview = true;
                previewShape.StartPoint = currentStart;
                  previewShape.EndPoint = currentEnd;
               previewShape.OutlinePen.DashStyle = DashStyle.Dash;
                        previewShape.Show(g);
                }
            }
        public void SelectShapeType(Descriptor type)
            {
                currentShapeType = type;
            }
            public void Dispose()
            {
            for(int i = shapeCount - 1; i >= 0; i--)
                {
                    shapes[i].Dispose();
                }
            }
          public Shape GetShape(int index)
            {
          if(index < shapeCount && index >= 0)
                {
                    return shapes[index];
                }
```

```
        throw new ArgumentOutOfRangeException(nameof(index), index , " is
                        out of bounds");
                                }
                    public void SelectShape(int index)
                                {
                                  selectedRow = index;
                                }
                    public void DeleteShape(int index)
                                {
                     if(index >= 0 && index < shapeCount)
                                    {
                                  shapes[index].Dispose();
                   for(int i = index; i < shapeCount - 1 ; i++)
                                      {
                                    shapes[i] = shapes[i+1];
                                      }
                                shapeCount = shapeCount - 1;
                                     selectedRow = null;
                                      }
                                }
                     public bool AddShape(Shape shape)
                                {
                                if(shapeCount >= shapes.Length)
                                    {
                                  return false;
                                    }
                                  shapes[shapeCount] = shape;
```

```
                shapeCount++;
                return true;
                }
            }
```

Form1

```
namespace Lab5;
public partial class Form1 : Form
{
    private Editor editor;
    private ObjectsTableWindow? objectsTableWindow;
    private Descriptor currentShapeType = ShapesDescriptor.Descriptors[0];
    private void SelectShapeType(Descriptor type)
    {
        Text = "Lab5 - " + type.Name;
        editor.SelectShapeType(type);
    }
    private void CreateMenu()
    {
        MenuStrip menuStrip = new MenuStrip();
        ToolStripMenuItem fileMenu = new ToolStripMenuItem("Файл");
        ToolStripMenuItem objectsMenu = new ToolStripMenuItem("Об'єкти");
        ToolStripMenuItem helpMenu = new ToolStripMenuItem("Довідка");
        ToolStripMenuItem tableMenu = new ToolStripMenuItem("Таблиця");

        ToolStripMenuItem saveTable = new ToolStripMenuItem("Зберегти
            Таблицю");
```

```
        ToolStripMenuItem loadTable = new ToolStripMenuItem("Завантажити
                            Таблицю");


                        Controls.Add(menuStrip);

                       MainMenuStrip = menuStrip;

                     menuStrip.Items.Add(fileMenu);

                    menuStrip.Items.Add(objectsMenu);

                     menuStrip.Items.Add(helpMenu);

                    menuStrip.Items.Add(tableMenu);

                  fileMenu.DropDownItems.Add(saveTable);

                  fileMenu.DropDownItems.Add(loadTable);


                     helpMenu.Click += HelpMenu_Click;

                   tableMenu.Click += TableMenu_Click;

                    saveTable.Click += SaveMenu_Click;

                   loadTable.Click += LoadTable_Click;


        foreach (Descriptor descriptor in ShapesDescriptor.Descriptors)

                                    {

                        ToolStripMenuItem item = new
                ToolStripMenuItem(descriptor.Name);

                            item.Tag = descriptor;

                     item.Click += ShapeSelector_Click;

                    objectsMenu.DropDownItems.Add(item);

                                    }

                                }

                  private void CreateShapesBar()

                                {
```

```
                ToolStrip shapesBar = new ToolStrip();
            shapesBar.ImageScalingSize = new Size(24, 24);

                    Controls.Add(shapesBar);

        foreach (Descriptor descriptor in ShapesDescriptor.Descriptors)
                                {
            ToolStripButton item = new ToolStripButton(descriptor.Name);
                item.DisplayStyle = ToolStripItemDisplayStyle.Image;
                    item.ToolTipText = descriptor.Name;
                        item.Tag = descriptor;
                    item.Image = descriptor.Icon;
                    item.Click += ShapeSelector_Click;
                        shapesBar.Items.Add(item);
                                }
                            }
                        public Form1()
                                {
                        InitializeComponent();
                    ClientSize = new Size(1080, 720);
                    this.DoubleBuffered = true;
                        editor = Editor.Instance;
                            CreateShapesBar();
                              CreateMenu();
                    SelectShapeType(currentShapeType);

                    this.MouseDown += Form_MouseDown;
```

```
                this.MouseMove += Form_MouseMove;
                  this.MouseUp += Form_MouseUp;
                    this.Paint += Form_Paint;
                 this.FormClosed += Form_Closed;
                                }
    private void Form_MouseDown(object? sender, MouseEventArgs e)
                                {
                  editor.OnMouseDown(e.Location);
                                }
    private void Form_MouseMove(object? sender, MouseEventArgs e)
                                {
                if (editor.OnMouseMove(e.Location))
                                  {
                               Invalidate();
                                  }
                                }
     private void Form_MouseUp(object? sender, MouseEventArgs e)
                                {
                    switch (editor.OnMouseUp())
                                  {
                case Editor.OnMouseUpResult.ShapeAdded:
                                Invalidate();
                               UpdateTable();
                                  break;
               case Editor.OnMouseUpResult.ArrayIsFull:
                 MessageBox.Show("Shapes Array is full");
                                Invalidate();
```

```
                                    break;
                                }
                            }
        private void Form_Paint(object? sender, PaintEventArgs e)
                            {
                        Graphics g = e.Graphics;
                         editor.DrawShapes(g);
                            }
    private void ShapeSelector_Click(object? sender, EventArgs e)
                            {
              ToolStripItem item = (ToolStripItem)sender!;
                Descriptor type = (Descriptor)item.Tag!;
                        SelectShapeType(type);
                            }
        private void HelpMenu_Click(object? sender, EventArgs e)
                            {
      MessageBox.Show("Choose Objects option to draw some shapes");
                            }
        private void TableMenu_Click(object? sender, EventArgs e)
                            {
    if( objectsTableWindow == null || objectsTableWindow.IsDisposed)
                                {
                objectsTableWindow = new ObjectsTableWindow();
                  objectsTableWindow.UpdateRows(GetRows());
              objectsTableWindow.SelectRow += OnTableRowSelected;
              objectsTableWindow.DeleteRow += OnTableRowDeleted;
                        objectsTableWindow.Show(this);
```

```
                }
                else
                {
                objectsTableWindow.Close();
                }
            }
        private void Form_Closed(object? sender, FormClosedEventArgs e)
            {
                editor.Dispose();
            }
        private List<TableRow> GetRows()
            {
            List<TableRow> result = new List<TableRow>();
            for(int i = 0; i < Editor.Instance.ShapeCount; i++)
                {
                Shape shape = Editor.Instance.GetShape(i);
                int startX = shape.StartPoint.X;
                int startY = shape.StartPoint.Y;
                int endX = shape.EndPoint.X;
                int endY = shape.EndPoint.Y;
                string name;
                Descriptor? descriptor =
        ShapesDescriptor.SearchDescriptorByKey(shape.Key);
                if(descriptor != null)
                {
                name = descriptor.Name;
                }
```

```
                                        else
                                        {
                                    name = "unknown Shape";
                                        }
            result.Add(new TableRow(startX, startY, endX, endY, name));
                                    }
                                return result;
                                    }
                            private void UpdateTable()
                                    {
        if(objectsTableWindow != null && !objectsTableWindow.IsDisposed)
                                        {
                        objectsTableWindow.UpdateRows(GetRows());
                                        }
                                    }
                private List<string> RowsToCsv(List<TableRow> rows)
                                    {
                        List<string> lines = new List<string>();
                            foreach (TableRow row in rows)
                                        {

lines.Add($"{row.Name},{row.StartX},{row.StartY},{row.EndX},{row.EndY}");
                                        }
                                    return lines;
                                    }
          private void SaveMenu_Click(object? sender, EventArgs e)
                                    {
```

```
            using(SaveFileDialog dialog = new SaveFileDialog())
                                    {
                    dialog.Filter = "CSV files (*.csv)|*.csv";
                     dialog.FileName = "shapesInfoReport.csv";


                    if(dialog.ShowDialog() == DialogResult.OK)
                                      {
                 File.WriteAllLines(dialog.FileName, RowsToCsv(GetRows()));
                                      }
                                    }
                                  }
              private void OnTableRowSelected(int index)
                                    {
                           editor.SelectShape(index);
                                 Invalidate();
                                    }
               private void OnTableRowDeleted(int index)
                                    {
                           editor.DeleteShape(index);
                                UpdateTable();
                                 Invalidate();
                                    }
        private void LoadTable_Click(object? sender, EventArgs e)
                                    {
            using (OpenFileDialog dialog = new OpenFileDialog())
                                      {
                    dialog.Filter = "CSV files (*.csv)|*.csv";
```

```
                    if (dialog.ShowDialog() != DialogResult.OK)
                                        {
                                       return;
                                        }
                string[] lines = File.ReadAllLines(dialog.FileName);
                            foreach(string line in lines)
                                        {
                            string[] parts = line.Split(',');
                                  if(parts.Length != 5)
                                            {
                                        continue;
                                            }
                                Descriptor? descriptor =
            ShapesDescriptor.SearchDescriptorByName(parts[0]);
                                if(descriptor == null)
                                            {
                                        continue;
                                            }
                        if(!int.TryParse(parts[1], out int x1) ||
!int.TryParse(parts[2], out int y1) || !int.TryParse(parts[3], out int x2)
                    || !int.TryParse(parts[4], out int y2))
                                            {
                                        continue;
                                            }
                            Shape shape = descriptor.Create();
                                shape.Key = descriptor.Key;
                            shape.StartPoint = new Point(x1, y1);
                             shape.EndPoint = new Point(x2, y2);
```

```
                    if (!editor.AddShape(shape))
                                {
                            shape.Dispose();
                        MessageBox.Show("SHAPE ARRAY IS FULL");
                                break;
                                }
                               }
                           UpdateTable();
                           Invalidate();
                               }
                              }
```

IconShapes

```
                    namespace Lab5;
                static class IconShapes
                            {
            public static Bitmap CreatePointIcon()
                            {
               Bitmap pointBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(pointBmp))
        using (Brush brush = new SolidBrush(Color.Black))
                               {
                    g.FillEllipse(brush, 8, 8, 8, 8);
                            }
                        return pointBmp;
                            }
             public static Bitmap CreateRectIcon()
                            {
```

```
            Bitmap rectBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(rectBmp))
            using (Pen pen = new Pen(Color.Black))
                                {
                    g.DrawRectangle(pen, 4, 6, 16, 12);
                                }
                            return rectBmp;
                                }
          public static Bitmap CreateEllipseIcon()
                                {
            Bitmap ellipseBmp = new Bitmap(24,24);
       using (Graphics g = Graphics.FromImage(ellipseBmp))
            using (Pen pen = new Pen(Color.Black))
                                {
                    g.DrawEllipse(pen, 2, 5, 20, 14);
                                }
                        return ellipseBmp;
                                }
        public static Bitmap CreateLineIcon()
                                {
            Bitmap lineBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(lineBmp))
            using (Pen pen = new Pen(Color.Black))
                                {
                    g.DrawLine(pen, 3, 3, 20, 20);
                                }
                            return lineBmp;
```

```
                                    }

                            public static Bitmap
   CreateALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineIcon()

                                    {

        Bitmap aLineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineBmp
                            = new Bitmap(24,24);

                                using (Graphics g =
Graphics.FromImage(aLineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLine
                                    Bmp))

                        using (Pen pen = new Pen(Color.Black))

                                        {

                                g.DrawLine(pen, 3, 3, 20, 20);

                                g.DrawEllipse(pen, 0, 0, 6, 6);

                                g.DrawEllipse(pen, 17, 17, 6, 6);

                                        }

       return aLineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineBmp;

                                    }

                    public static Bitmap CreateCubeIcon()

                                    {

                        Bitmap CubeBmp = new Bitmap(24,24);

                    using(Graphics g = Graphics.FromImage(CubeBmp))

                        using(Pen pen = new Pen(Color.Black))

                                        {

                            g.DrawRectangle(pen, 0, 6, 12, 12);

                            g.DrawRectangle(pen, 6, 0, 12, 12);

                               g.DrawLine(pen, 0, 6, 6, 0);

                              g.DrawLine(pen, 12, 6, 18, 0);

                              g.DrawLine(pen, 0, 18, 6, 12);
```

```
                    g.DrawLine(pen, 12, 18, 18, 12);
                }
            return CubeBmp;
        }
    }
```

ObjectsTableWindow

```
namespace Lab5;
public class ObjectsTableWindow : Form
{
    private DataGridView grid = new DataGridView();
    public event Action<int>? SelectRow; // Bonus 1
    public event Action<int>? DeleteRow; // Bonus 2
    public bool IsUpdating = false;
    public ObjectsTableWindow()
    {
        grid.Columns.Add("name", "Назва");
        grid.Columns.Add("startX", "StartX");
        grid.Columns.Add("startY", "StartY");
        grid.Columns.Add("endX", "EndX");
        grid.Columns.Add("endY", "EndY");
        grid.ReadOnly = true;
        grid.AllowUserToAddRows = false;
        grid.AllowUserToDeleteRows = false;
        grid.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
        grid.MultiSelect = false;
        grid.SelectionChanged += Grid_SelectionChanged;
        grid.Dock = DockStyle.Fill;
```

```
                grid.KeyDown += Grid_KeyDown;

                Controls.Add(grid);

            ClientSize = new Size(540, 360);

                Text = "Objects Table";

                }

        public void UpdateRows(List<TableRow> rows)

                {

                IsUpdating = true;

                grid.Rows.Clear();

            foreach (TableRow row in rows)

                    {

    grid.Rows.Add(row.Name, row.StartX, row.StartY, row.EndX,
                row.EndY);

                    }

                grid.ClearSelection();

                IsUpdating = false;

                }

  private void Grid_SelectionChanged(object? sender, EventArgs e)

                {

                if(IsUpdating) return;

            if(grid.CurrentRow != null)

                    {

          SelectRow?.Invoke(grid.CurrentRow.Index);

                    }

                }

   private void Grid_KeyDown(object? sender, KeyEventArgs e)

                {
```

```
            if(e.KeyCode == Keys.Delete && grid.CurrentRow != null)
                {
                DeleteRow?.Invoke(grid.CurrentRow.Index);
                }
            }
        }
```

ShapeDescriptor

```
namespace Lab5;

public static class ShapesDescriptor
{
    public static readonly List<Descriptor> Descriptors = new
    List<Descriptor>
    {
        new Descriptor("Крапка" , "Point", IconShapes.CreatePointIcon(),
        () => new PointShape()),
        new Descriptor("Лінія" , "Line", IconShapes.CreateLineIcon(), ()
        => new LineShape()),
        new Descriptor("Прямокутник", "Rectangle",
        IconShapes.CreateRectIcon(), () => new RectShape()),
        new Descriptor("Еліпс", "Ellipse", IconShapes.CreateEllipseIcon(),
        () => new EllipseShape()),
        new Descriptor("Лінія з еліпсами у формі кола на обох кінцях цієї
    лінії", "ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLine",
IconShapes.CreateALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineIc
on(), () => new
    ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape()),
        new Descriptor("Куб", "Cube", IconShapes.CreateCubeIcon(), () =>
        new CubeShape())
    };
    public static Descriptor? SearchDescriptorByKey(string? key)
```

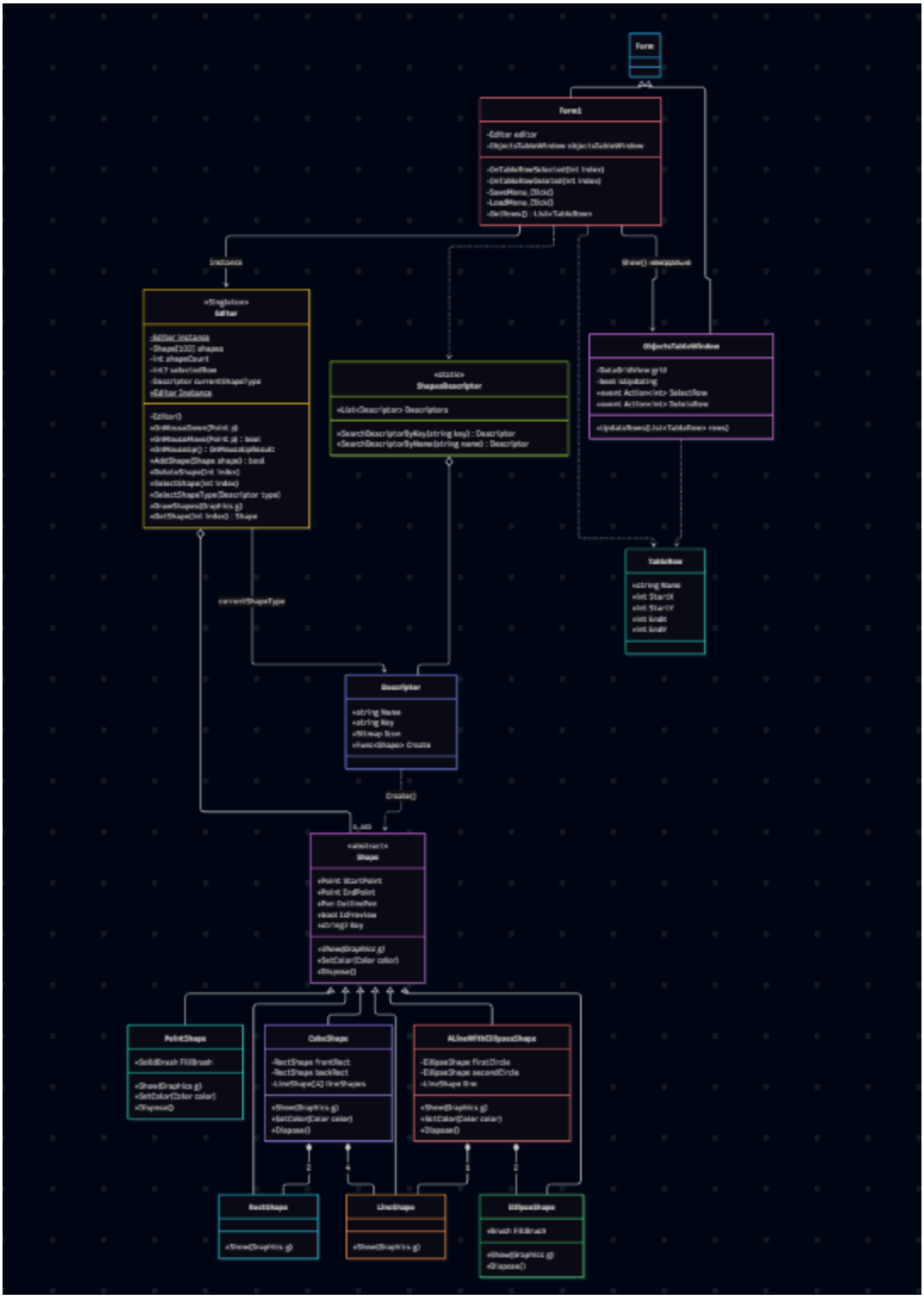
```
                {
            foreach(Descriptor descriptor in Descriptors)
                {
                if (descriptor.Key == key)
                    {
                    return descriptor;
                    }
                }
            return null;
            }
        public static Descriptor? SearchDescriptorByName(string? name)
            {
            foreach(Descriptor descriptor in Descriptors)
                {
                if (descriptor.Name == name)
                    {
                    return descriptor;
                    }
                }
            return null;
            }
        }
```

TableRow

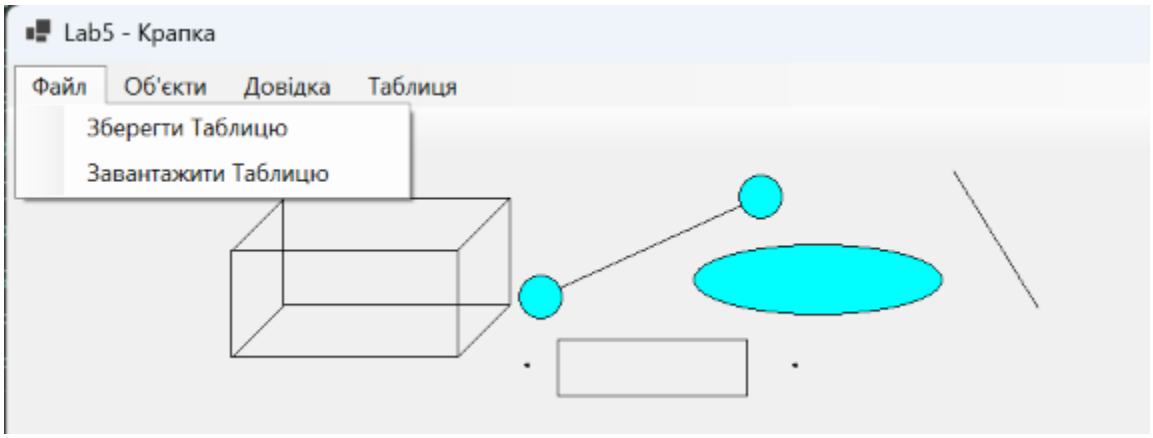
```
namespace Lab5;
public class TableRow
    {
    public int StartX {get;}
```

```
                public int StartY {get;}
                public int EndX {get;}
                public int EndY {get;}
                public string Name {get;}
    public TableRow(int startX, int startY, int endX, int endY, string
                name)
                    {
                StartX = startX;
                StartY = startY;
                  EndX = endX;
                  EndY = endY;
                  Name = name;
                    }
                  }
```

Діаграма


Form
Form1
-Editor editor
-ObjectsTableWindow objectsTableWindow
-OnTableRowSelected(int index)
-OnTableRowDeselected(int index)
-SaveMenu_Click()
-LoadMenu_Click()
-GetRows() : List<TableRow>
Instance
Show() «exception»
«Singleton»
Editor
-Editor instance
-Shape[,][] shapes
-int shapeCount
-int? selectedRow
-Descriptor currentShapeType
+Editor Instance
-Editor()
+OnMouseDown(Point p)
+OnMouseMove(Point p) : bool
+OnMouseUp() : OnMouseUpResult
+AddShape(Shape shape) : bool
+RotateShape(int index)
+SelectShape(int index)
+SelectShapeType(Descriptor type)
+DrawShapes(Graphics g)
+GetShape(int index) : Shape
«static»
ShapeDescriptor
+List<Descriptor> Descriptors
+SearchDescriptorByKey(string key) : Descriptor
+SearchDescriptorByName(string name) : Descriptor
ObjectsTableWindow
-DataGridView grid
-bool isUpdating
+event Action<int> SelectRow
+event Action<int> DeleteRow
+UpdateRows(List<TableRow> rows)
TableRow
+string Name
+int StartX
+int StartY
+int EndX
+int EndY
currentShapeType
Descriptor
+string Name
+string Key
+Bitmap Icon
+Func<Shape> Create
Create()
1..*
«abstract»
Shape
+Point StartPoint
+Point EndPoint
#Pen OutlinePen
+bool IsPreview
+string? Key
+Show(Graphics g)
+SetColor(Color color)
+Dispose()
PointShape
+SolidBrush FillBrush
+Show(Graphics g)
+SetColor(Color color)
+Dispose()
CubeShape
-RectShape frontRect
-RectShape backRect
-LineShape[4] lineShapes
+Show(Graphics g)
+SetColor(Color color)
+Dispose()
ALineWithEllipseShape
-EllipseShape firstCircle
-EllipseShape secondCircle
-LineShape line
+Show(Graphics g)
+SetColor(Color color)
+Dispose()
2
4
3
2
RectShape
+Show(Graphics g)
LineShape
+Show(Graphics g)
EllipseShape
+Brush FillBrush
+Show(Graphics g)
+Dispose()

Скріншоти



Висновки:

У ході роботи я навчився реалізовувати патерн Singleton та організовувати взаємодію між вікнами через події, не створюючи залежностей між модулями. Також я зрозумів, як працює поліморфізм на практиці, і навчився зберігати та завантажувати дані з файлу, перевіряючи їх на коректність.